LUCENE VS MARIA DB FTS

TECHNICAL EVALUATION AND RECOMMENDATION

Prashant Hariharan

REQUIREMENTS

1 Fair evaluation

Compare Lucene and MariaDB FTS under the same functional scope and test conditions.

2 Same data, same rules

Use one application, one request model, and one shared dataset to avoid biased results.

3 Measure what matters

Assess relevance features, hit accuracy, p50 latency, and p95 latency.

4 Actionable outcome

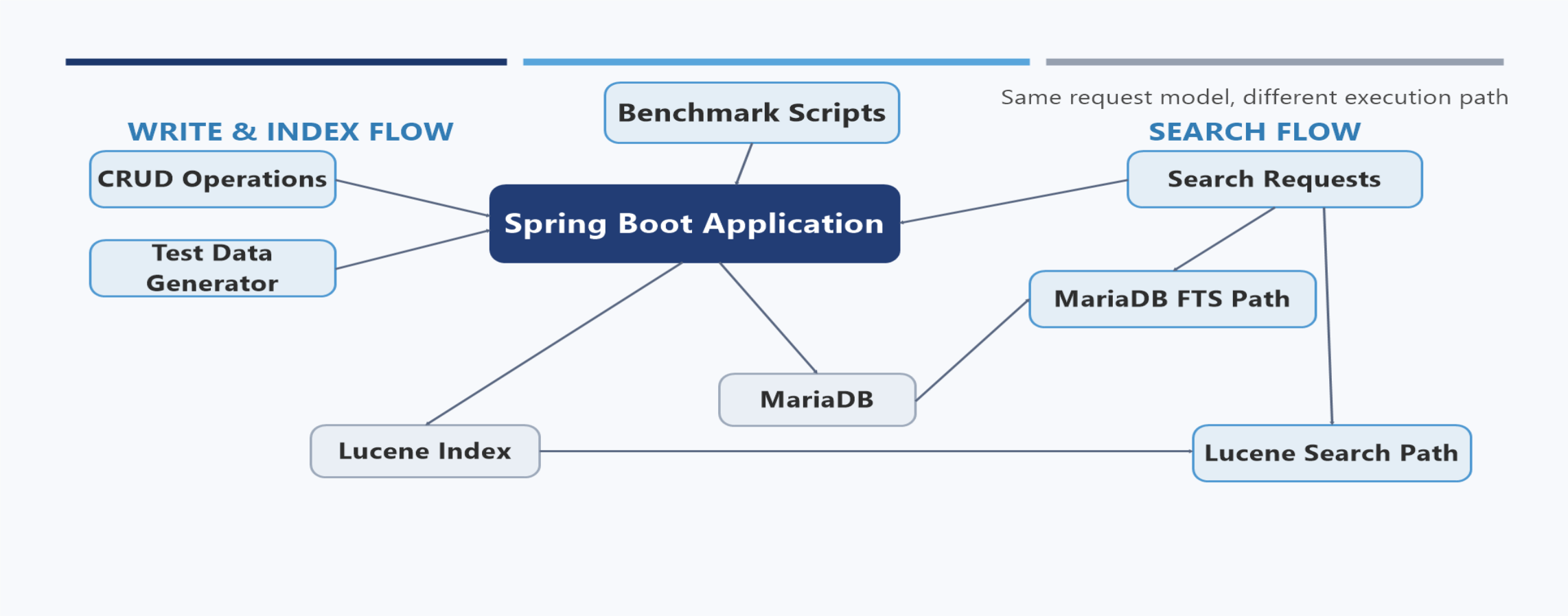
End with a clear recommendation based on capability, speed, and future fit.

TECH STACK

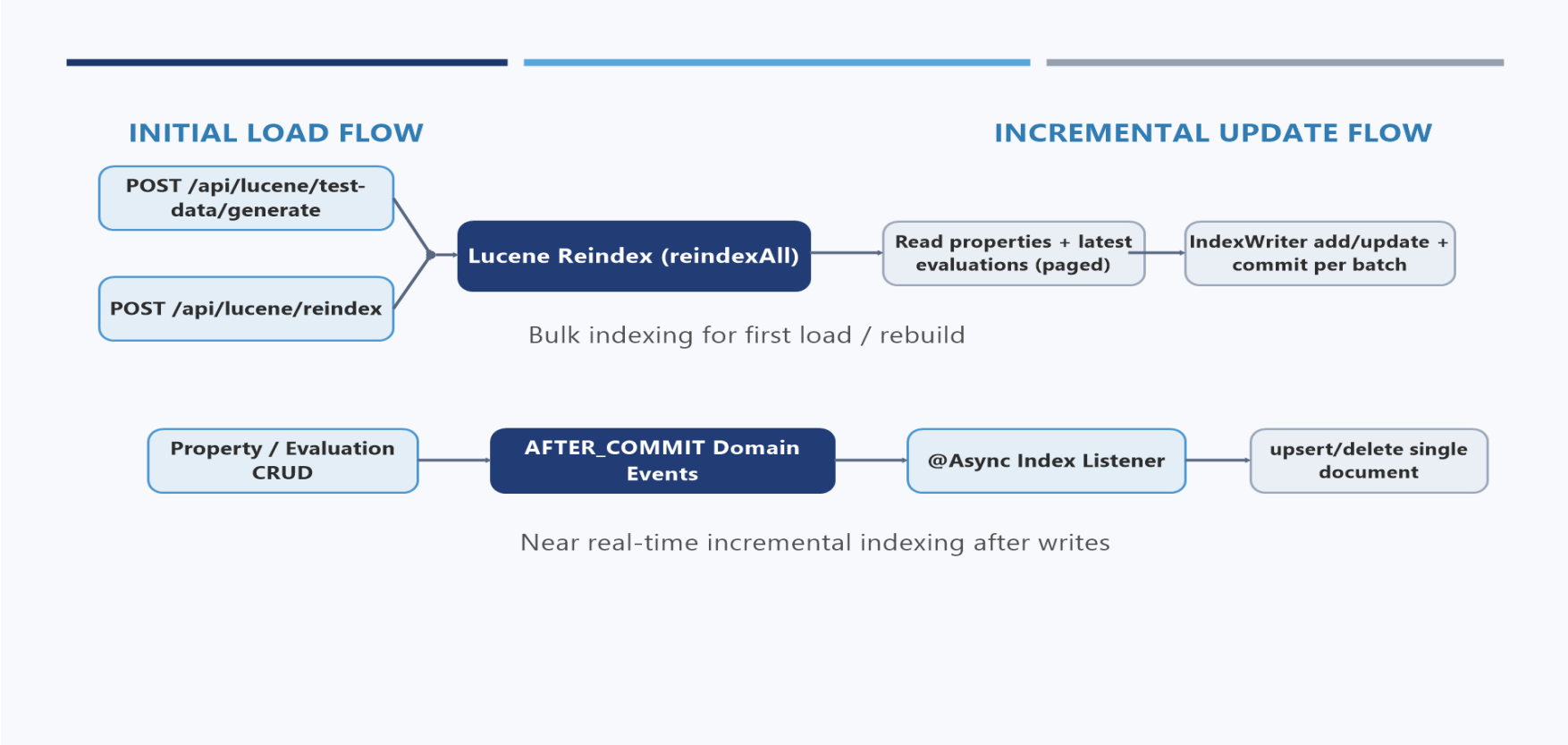
Core Tech stack

- Java 21
- Spring Boot 4.0.3
- Apache Lucene 9.11.1
- MariaDB 11.0.6
- Liquibase
- Maven 3.9+
- Docker - Docker Compose

HIGH LEVEL ARCHITECTURE



LUCENE INDEX UPDATE LIFECYCLE



FEATURE COMPARISON AND BENCHMARKING

Features Being Compared

- Basic full-text search across title and description
- Synonym search, for example flat/apartment and garage/parking
- Fuzzy search for typo tolerance, for example Berln to Berlin
- Score boosting to prioritize exact matches and more important fields
- Geo Radius based queries. Eg 5 kms from Berlin

Benchmarking

- Run the benchmark script against both endpoints with the same scenarios and data set
- Capture p50 and p95 latency and export CSV, JSON and Excel results
- Validate both response time and expected hit counts

```
Example: powershell -ExecutionPolicy Bypass -File benchmarking/run-benchmark.ps1
-Iterations 30 -Warmup 5
```

TEST DATA GENERATOR

- Provides a repeatable POST endpoint to create benchmark-ready synthetic properties and evaluations in batches.
- The same generated dataset can be used for both Lucene and MariaDB FTS, ensuring a fair side-by-side comparison.
- Adds deterministic comparison fixtures (synonyms/typos/field-boost scenarios/geo point) as part of normal generated data
- Can optionally trigger Lucene reindex after generation
- By default it produces around 200000 properties & evaluations

Recommended demo flow: generate data with reindexAfterGeneration=false, then call POST /api/lucene/reindex separately.

LIVE DEMO

MARIADB FTS VS LUCENE: PRACTICAL TRADE-OFFS

Practical comparison from the live demo perspective.

Feature	MariaDB Full-Text Search	Apache Lucene	Winner
Setup & operations	Simple. One datastore, fewer moving parts.	More moving parts. Separate index lifecycle.	**MariaDB FTS**
Transactional consistency	Strong , as there is no need of explicit sync.	Needs indexing / synchronization discipline.	**MariaDB FTS**
Basic full-text search	Good enough for baseline keyword search and filters.	Also strong, with better control over ranking.	**Both**
Synonym search	Weak / not natural in the POC.	Strong via analyzer pipeline and synonym filter.	**Lucene**
Fuzzy / typo tolerance	Limited and cumbersome.	Strong typo tolerance in the live demo.	**Lucene**
Score boosting	Limited ranking control.	Purpose-built scoring and query boosting support.	**Lucene**
Advanced query composition	Native SQL gets harder to read and maintain.	Richer search DSL / API model.	**Lucene**

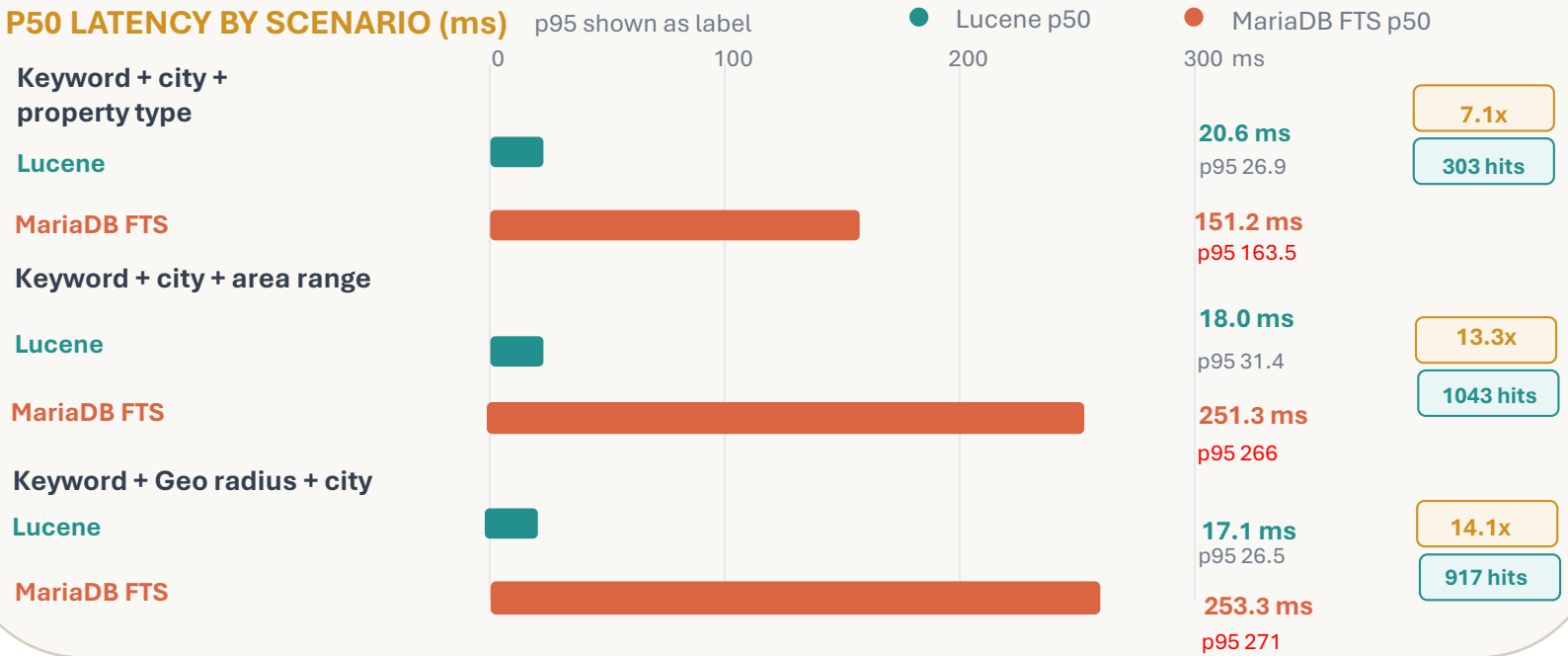
BENCHMARK RESULTS

Lucene achieved 7.1x–14.1x lower median latency with equal hit counts across all benchmark scenarios.

3 / 3
Scenarios - Lucene had lower p50 latency

7.1x – 14.1x
median latency improvement vs MariaDB FTS

P50 LATENCY BY SCENARIO (ms)



Equal hits (3/3)

Key Findings
- Lower p50 in every measured scenario
- Lower p95 tail latency in both cases
- Observed ranges also stayed materially lower

Test Conditions
- 30 measured runs, 5 warmup runs
- Same request payloads and same app instance
- POST API timing at endpoint level
- Preloaded MariaDB data and existing Lucene index

Scope Notes
- Single-user sequential benchmark
- No cold-cache or concurrency test yet

Source: benchmark-20260326-092700.csv and run-benchmark.ps1 | 30 measured iterations, 5 warmup | benchmark date: 2026-03-26 | endpoints: /api/lucene/search and /api/mariadb-fts/search

MANAGEMENT SUMMARY

Decision and implications from the POC benchmark and capability evaluation.

Recommendation: keep Lucene as the primary search engine for this use case; do not replace it with MariaDB FTS.

Evidence from the POC

- Faster in every measured scenario
- 7.1x-14.1x lower median latency
- Equal hit counts in the compared runs

Why Lucene is the safer choice

- Native synonyms, typo tolerance, and boosting
- Cleaner relevance tuning through analyzers
- Less SQL complexity as search grows

Near-term operating model

- MariaDB remains the system of record
- Lucene remains the search layer

Bottom line: MariaDB FTS is acceptable for simple embedded keyword search, but Lucene is the better fit when relevance and performance are product-critical.

OPENSEARCH AS A STRATEGIC ALTERNATIVE

Not the immediate recommendation, but the clearest upgrade path when search becomes distributed.

Use OpenSearch when distributed scale becomes necessary

When standalone Lucene is enough

- Single application or single-node deployment
- Direct control of indexing and ranking
- No immediate need for cluster operations

Signals to evaluate OpenSearch

- Multi-node indexing or search
- High availability, replication, or disaster recovery
- Need for lifecycle, observability, security, or richer APIs

What OpenSearch adds

- Shards, replicas, and cluster coordination
- Node discovery, snapshots, and rebalancing
- Operational tooling that reduces custom code

Recommendation for now: keep the Lucene-based design for the current scope; treat OpenSearch as a future platform decision, not a prerequisite for this POC.

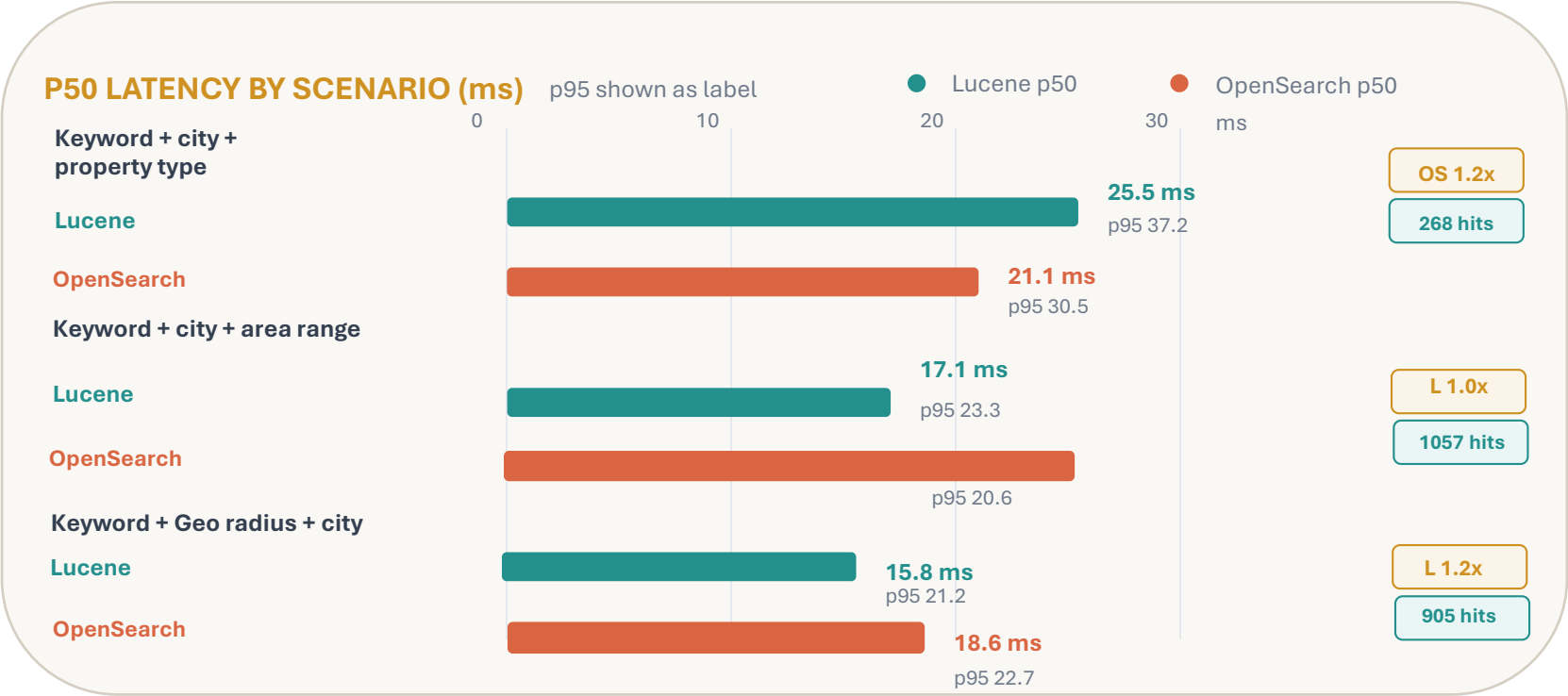
BENCHMARK RESULTS

Lucene and OpenSearch stayed close on median latency, with equal hit counts across all three benchmark scenarios.

2 / 3
Scenarios - Lucene had lower p50 latency

1.0x - 1.2x
median p50 gap across Lucene vs OpenSearch

Equal hits (3/3)



Key Findings

- Close p50 latency in every scenario
- Lucene lower p50 in 2 scenarios
- Equal hit counts across all 3 runs

Test Conditions

- 30 measured runs, 5 warmup runs
- Same payloads and same app instance
- POST API timing at endpoint level
- Indexed data loaded before timing

Scope Notes

- Single-user sequential benchmark
- No cold-cache or concurrency test yet

Source: benchmark-20260419-154239.csv and run-benchmark.ps1 | 30 measured iterations, 5 warmup | benchmark date: 2026-04-19 | endpoints: /api/lucene/search and /api/opensearch/search

MINI MIGRATION STRATEGY

Move in phases, keep rollback simple, and prove relevance plus latency before full cutover.

Build OpenSearch in parallel, validate against Lucene, then shift traffic gradually.

Phase 1: Prepare

- Freeze Lucene query behavior as the baseline
- Define matching mappings, analyzers, and ranking
- Build a repeatable full reindex from MariaDB

Phase 2: Parallel run

- Keep Lucene live while OpenSearch indexes in parallel
- Dual-write changed data to both indexes
- Compare hits, ranking quality, p50, and p95

Phase 3: Controlled cutover

- Shift a small share of reads via feature flag or canary
- Monitor relevance gaps, failures, latency, and index lag
- Keep Lucene rollback until OpenSearch is stable

Migration principle: MariaDB stays the source of truth, indexing runs in parallel during transition, and cutover remains reversible.

QUESTIONS?

THANK YOU

PRASHANT HARIHARAN